SmartCards

Alkalmazás

Teljes Dokumentáció

Verzió: 1.0

Készítették: Laczkó István, Brückner Gábor

GitHub link:

Tartalomjegyzék

1. Bevezetés....4
2. A projekt célja, célközönsége, hatóköre és általános áttekintése....4
I. A projekt célja....4
II. Célközönség....5
III. A projekt hatóköre....5
IV. Általános áttekintés....6
3. Specifikáció....6
I. Architekturális és technikai alapok....6
II. Felhasználói fiókok és jogosultságok....7
III. Fejlesztési módszertan és Csapatmunka....7
4. Rendszeráttekintés....10
I. Architektúra bemutatása....10
5. Használati útmutató....11
I. Felhasználói felület áttekintése....11
II. Főbb funkciók használata és Példák....11
6. REST API Dokumentáció....16
I. Hitelesítés (AccountController)....16
II. Paklik Kezelése (DecksController) *Autentikáció szükséges*....16
III. Kártyák Kezelése (CardsController)....17
IV. Statisztikák (StatsController)....17
V. Tanulási Statisztikák Matematikai és Logikai Összegzése....17
7. Frontend Komponensek....19

I. Fő Oldalak:.....19
II. Kártya mechanika és vizuális interakció.....20
III. A folyamatos adattárolás és betöltés:.....21
IV. Globális Állapotok és UI testreszabhatóság.....22
8. Részletes Tesztelési Stratégia.....23
I. Manuális Tesztek és Várt Eredmények.....23
II. Automatizált Tesztek.....24
9. Adatbázis Dokumentáció.....25
.....25
I. Adatbázis Modell és ER Diagram.....26
10. Adatbázis Séma és Kapcsolatok.....26
I. AppUser (Felhasználói fiók).....27
II. Deck (Kártyacsomag / Pakli).....27
III. Card (Tanulókártya / Flashcard).....28
IV. UserStats (Globális Felhasználói Statisztika).....29
V. DeckStats (Pakli specifikus statisztikák).....29
VI. CardStats (Kártya specifikus tanulási állapot).....30
11. Tesztelés.....31
VII. Egységtesztek áttekintése és példák.....32
VIII. Manuális és Automatizált Tesztesetek, Várt Eredmények.....33
12. Jövőbeli fejlesztési lehetőségek.....35
I. Funkcionális fejlesztési lehetőségek.....35
II. Nem funkcionális fejlesztési lehetőségek.....36
13. Telepítési és Üzemeltetési Útmutató.....37
I. Backend indítása:.....37
II. Frontend indítása:.....37
14. Felhasznált források.....38
I. Hivatkozások és Linkek.....38
15. Szerzői jogi nyilatkozat és Licenc.....40

1. Bevezetés

A digitális oktatás és az önálló tanulás szerepe az elmúlt években robbanásszerűen megnőtt. A diákok, nyelvtanulók és a folyamatosan fejlődni vágyó szakemberek mind olyan eszközöket keresnek, amelyekkel a tanulásra szánt időt a lehető leghatékonyabban tudják felhasználni. A hagyományos, papíralapú tanulókártyák (flashcards) régóta bizonyítottan az egyik leghatékonyabb eszközt jelentik a tények, szavak, fogalmak elsajátítására, mivel az aktív felidézés (active recall) elvére épülnek.

A **SmartCards** projekt ezt a jól bevált módszertant ülteti át egy modern, felhőalapú, interaktív webes környezetbe. Az alkalmazás célja, hogy kiváltsa a fizikai kártyákat, miközben olyan digitális előnyőket biztosít, mint az adatok adatbázisban történő biztonságos tárolása, a tanulási statisztikák folyamatos és precíz követése, valamint a kártyák és kártyacsomagok (paklik) könnyű szerkeszthetősége és rendszerezése.

A projekt a szoftverfejlesztési vizsgamunka részeként készült, ötvözve a modern backend fejlesztési elveket (C# .NET 10) és a korszerű, reszponzív frontend technológiákat, demonstrálva a teljes körű (full-stack) fejlesztői kompetenciákat.

2. A projekt célja, célközönsége, hatóköre és általános áttekintése

I. A projekt célja

A **SmartCards** elsődleges célja egy olyan platform biztosítása, amely gamifikációs elemekkel (például tanulási sorozatok, ún. "streak"-ek követésével) és részletes vizuális statisztikákkal motiválja a felhasználókat a napi szintű, folyamatos tanulásra. A cél egy letisztult, figyelemelterelésmentes környezet megteremtése, ahol a

felhasználó a saját tempójában, a saját maga által összeállított anyagokból tanulhat.

II. Célközönség

Az alkalmazás széles rétegeket céloz meg:

- **Középiskolás és egyetemista diákok:** Akiknek nagy mennyiségű lexikális tudást (történelem évszámok, biológia fogalmak, anatómia) kell elsajátítaniuk.
- **Nyelvtanulók:** Akik az új szókincset és nyelvtani szabályokat szeretnék ismétléssel rögzíteni.
- **Szakemberek és élethosszig tanulók:** Például informatikusok, orvosok vagy jogászok, akik vizsgákra készülnek vagy szinten akarják tartani a tudásukat.

III. A projekt hatóköre

A projekt keretein belül megvalósítandó:

- Biztonságos felhasználói fiókkezelés és hitelesítés.
- Kártyacsomagok (Decks) és kártyák (Cards) létrehozása, olvasása, módosítása és törlése (CRUD).
- Interaktív tanulási/kvíz munkamenet, amely rögzíti a kártyák tudásszintjét.
- Részletes statisztika: Napi aktivitás mérése, tanulási széria számolása, paklik megtanultsági százalékának megjelenítése.
- Reszponzív, mobilbarát webes felület biztosítása.

A projekt keretein kívül eső:

- Valós idejű, többjátékos tanulási módok.
- Fizetési kapuk és prémium előfizetéses modellek integrációja.

- Natív iOS és Android mobilalkalmazások fejlesztése (az alkalmazás webes böngészőből dedikáltan használható).

IV. Általános áttekintés

A **SmartCards** egy egyoldalas webalklalmazásból (Single Page Application - SPA) és egy mögötte álló RESTful API-ból áll. A felhasználó a böngészőn keresztül lép interakcióba az alkalmazással. A regisztrációt és bejelentkezést követően egy vezérlőpultra érkezik, ahol láthatja az eheti aktivitását egy diagramon, a napi tanulási sorozatát, valamint a legutóbb használt kártyacsomagokat. Innen navigálhat a paklikezelőbe, ahol új kártyákat vehet fel, vagy elindíthat egy tanulási szekciót. A tanulás során a rendszer kártyákat dob fel, melyeknek először a kérdés oldalát látja, majd "fordítás" után ellenőrizheti a választ, és értékelheti a saját tudását. A rendszer a háttérben minden lépést naplóz, így a felhasználó mindig naprakész képet kap a fejlődéséről.

3. Specifikáció

I. Architekturális és technikai alapok

A specifikáció meghatározza a rendszer felépítésének technikai és strukturális kereteit.

- **Architekturális minta:** A projekt többrétegű (N-Tier) architekturális mintát követ. Ez magában foglalja a prezentációs réteget (Frontend), az API végpontokat biztosító Controller réteget, az üzleti logikát tartalmazó Service réteget, valamint az adatelérésért felelős Repository réteget.
- **Kommunikáció:** A kliens és a szerver közötti kommunikáció szabványos HTTP(S) protokollon keresztül történik JSON adatformátum használatával, REST konvenciók (GET, POST, PUT, DELETE) alapján.

- **Állapotkezelés:** A szerver állapotmentes (Stateless). A kérések hitelesítése JSON Web Token (JWT) segítségével történik, melyet a kliens az Authorization header-ben (Bearer token) küld el. A hosszú távú bejelentkezés fenntartását HTTP-only sütikben tárolt Refresh Tokenek biztosítják.
- **Adatbázis-kezelés:** A rendszer MySQL relációs adatbázist használ. Az objektum-relációs leképzést (ORM) az Entity Framework Core végzi, amely lehetővé teszi az adatbázis sémájának kód alapú (Code-First) menedszelését migrációk segítségével. A MySQL választása mellett a skálázhatóság és a széles körű közösségi támogatás miatt döntöttünk.
- **Felhasználói Felület:** A UI komponens-alapú megközelítéssel készül, amely lehetővé teszi a kód újrafelhasználhatóságát. A dizájn "Mobile First" elvek mentén épül fel, biztosítva a tökéletes megjelenést mind asztali, mind mobileszközökön.

II. Felhasználói fiókok és jogosultságok

A rendszer sajátossága, hogy nincsenek külön jogosultsági szintek vagy dedikált adminisztrátori szerepkörök. Az adatelérés teljes mértékben tulajdonosalapú.

- **Vendég (nem bejelentkezett) felhasználó:** Csak a nyílt végpontokat éri el (pl. regisztrációs és bejelentkezési felület). A rendszer többi funkciója rejtett számára.
- **Regisztrált felhasználó:** Minden bejelentkezett felhasználó azonos jogkörökkel rendelkezik. Egy teljesen szeparált, privát környezetben dolgozik. Kizárólag a saját maga által létrehozott paklikat, kártyákat és statisztikákat láthatja, illetve módosíthatja.

III. Fejlesztési módszertan és Csapatmunka

A **SmartCards** projekt fejlesztése során figyeltünk a hatékony csapatmunkára és az iparági standard szoftverfejlesztési folyamatok

betartására. A munka gördülékenységét az agilis (Agile) elvek és a modern kollaborációs eszközök ötvözése biztosította.

Használt eszközök és Agilis módszertan

Bár kétszemélyes csapatként dolgoztunk, a professzionális szoftverfejlesztéshez szükséges módszereket/tanulmányokat maradéktalanul alkalmaztuk:

- **Feladatkövetés (Trello-féle Kanban board):** Az agilis módszertanból a Kanban vizuális feladatkezelését emeltük át. A megvalósítandó funkciókat (Backlog) egy digitális táblán rögzítettük, és az állapotuknak mozgattuk őket. Ez biztosította az átláthatóságot, így mindkét csapattag pontosan tudta, hol tart a projekt, és mi a következő prioritás.
- **Verziókövetés (GitLab):** A forráskód biztonságos tárolására és a párhuzamos munkavégzés támogatására a GitLab rendszert használtuk. A Git branch-ek (ágak) alkalmazásával elkerütük a kódütközéseket: a backend és a frontend fejlesztése egymástól elszigetelten, mégis összehangoltan történhetett, majd a stabil verziókat rendszeresen integráltuk elsősorban a (developer) ágba majd a legvégén a kész projektet a fő (main) ágba.

Munkamegosztás és Szerepkörök

A feladatokat teljes szétválasztás alapján, az egyéni kompetenciákhoz igazítva osztottuk fel, így a kliens és szerveroldal fejlesztése párhuzamosan tudott haladni.

Brückner Gábor *Frontend Fejlesztő és* UI/UX specialista

- Felhasználói felület és élmény: A modern, reszponzív UI kialakítása Tailwind CSS és DaisyUI keretrendszerek segítségével, valamint a vizuális komponensstruktúra felépítése.
- Kliensoldali implementáció: Az Angular alapú kezelőfelület elemeinek megvalósítása és a szerverrel történő aszinkron kommunikáció (API hívások) leprogramozása.

- **Dokumentáció és Prezentáció:** A hivatalos szakmai dokumentáció összeállítása, a prezentációs anyagok kidolgozása és a projektadminisztráció kezelése.
- **Állapotkezelés és adatfolyamok:** Az RxJS könyvtár alkalmazása az aszinkron adatfolyamok és megfigyelhető objektumok (Observables) kezelésére, biztosítva a reaktív felhasználói élményt.
- **Dinamikus témakezelés:** Egyedi stílusrendszer és többféle választható vizuális téma implementálása, amely lehetővé teszi a felhasználó számára a felület személyre szabását.
- **Vizuális interakciók és animációk:** A tanulókártyák élethű megfordítási effektusainak, az oldalak közötti sima átmeneteknek és a felhasználói visszajelzések vizuális megjelenítésének kidolgozása.
- **Többnyelvűségi támogatás:** Az alkalmazás nemzetköziesítése (i18n), a nyelvi váltó és a dinamikus szövegbetöltés technikai megvalósítása.

Laczkó István *Fullstack Fejlesztő és Szoftverarchitekt*

- **Rendszerarchitektúra és tervezés:** A teljes szoftverarchitektúra megtervezése, az N-Tier (többrétegű) felépítés és a Clean Architecture elveinek érvényesítése.

- **Backend - Frontend Logika:** A robusztus .NET REST API kidolgozása, valamint a komplett játékmotor és tanulási logika (pl. statisztikai számítások, algoritmusok) backend-es és frontend-es implementálása.
- **Adatkezelés és CRUD:** Az adatbázis-séma megtervezése, valamint a paklik (Decks) és kártyák (Cards) teljes körű CRUD (létrehozás, olvasás, módosítás, törlés) műveleteinek és adatelérési rétegének kidolgozása.
- **Biztonság és Minőség:** A JWT-alapú hitelesítési rendszer, a biztonsági protokollok, valamint a rendszer stabilitását biztosító egységtesztelési folyamatok megvalósítása.
- **Technikai felügyelet:** A frontend és backend közötti integráció irányítása és a kódminőség ellenőrzése.

4. Rendszeráttekintés

I. Architektúra bemutatása

A **SmartCards** alkalmazás egy modern, szétválasztott kliens-szerver architektúrára épül, amely garantálja a rendszer rugalmasságát, gyorsaságát és skálázhatóságát.

- **Frontend (Kliens oldal):** Az alkalmazás felülete egy Single Page Application (SPA), amely javarészt komponensalapú megközelítéssel készült. Ez azt jelenti, hogy az oldalfrissítések nélkül, dinamikusan cserélődnek a képernyőn a tartalmak (pl. a tanulókártyák forgatása során), ami rendkívül gyors és folyékony felhasználói élményt (UX) biztosít. A kliens aszinkron HTTP kéréseken (REST) keresztül kommunikál a szerverrel.
- **Backend (Szerver oldal):** A szerveroldal egy C# .NET 10 alapú RESTful Web API. Az architektúra szigorúan **N-rétegű (N-Tier)** mintát követ:
- **Controller réteg:** Fogadja a kliens felől érkező HTTP kéréseket, validálja azokat, majd továbbítja az üzleti rétegnek.
- **Service réteg:** Itt kapott helyet az üzleti logika (például a tanulási statisztikák, a streak-ek napi frissítésének és a tudásszázalékok kalkulálásának komplex algoritmusa).
- **Repository réteg:** Az adatbázissal való közvetlen kommunikációért felel.
- **Adatbázis és Perzisztencia:** Az objektum-relációs leképzést (ORM) az Entity Framework Core végzi Code-First megközelítéssel. A fejlesztés során egy nagy teljesítményű relációs adatbázist (MySQL) használunk az adatok perzisztens tárolására.
- **Hitelesítés:** A rendszer állapotmentes (Stateless), a jogosultságkezelés szabványos JSON Web Token (JWT) technológiára épül.

5. Használati útmutató

I. Felhasználói felület áttekintése

A **SmartCards** felhasználói felülete a figyelemelterelés mentes tanulást helyezi előtérbe. A dizájn letisztult, és reszponzív módon igazodik a kijelző méretéhez (mobil, tablet, asztal). A bejelentkezés után a felhasználó a **Vezérlőpultra (Főoldalra)** érkezik. A navigációs sáv segítségével könnyedén válthat a főbb menüpontok között:

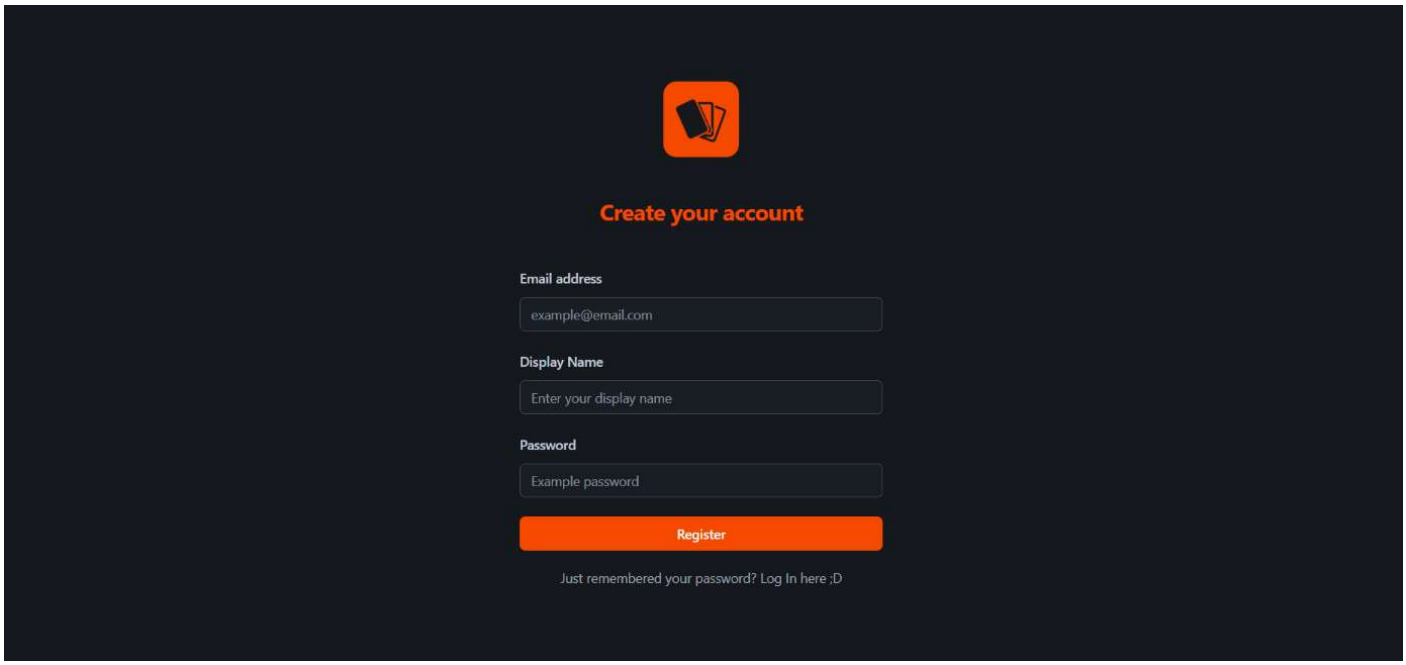
- **Főoldal:** Globális statisztikák, heti aktivitási grafikon és a tanulási széria (Streak) láng ikonja.
- **Paklik:** Itt találhatók a létrehozott kártyacsomagok és itt indítható a tanulás.
- **Rólunk:** A projekt leírása található meg, a vizsgakövetelmények egyik eleme
- **Profil:** Profiladatok és hitelesítési beállítások módosítása.

II. Főbb funkciók használata és Példák

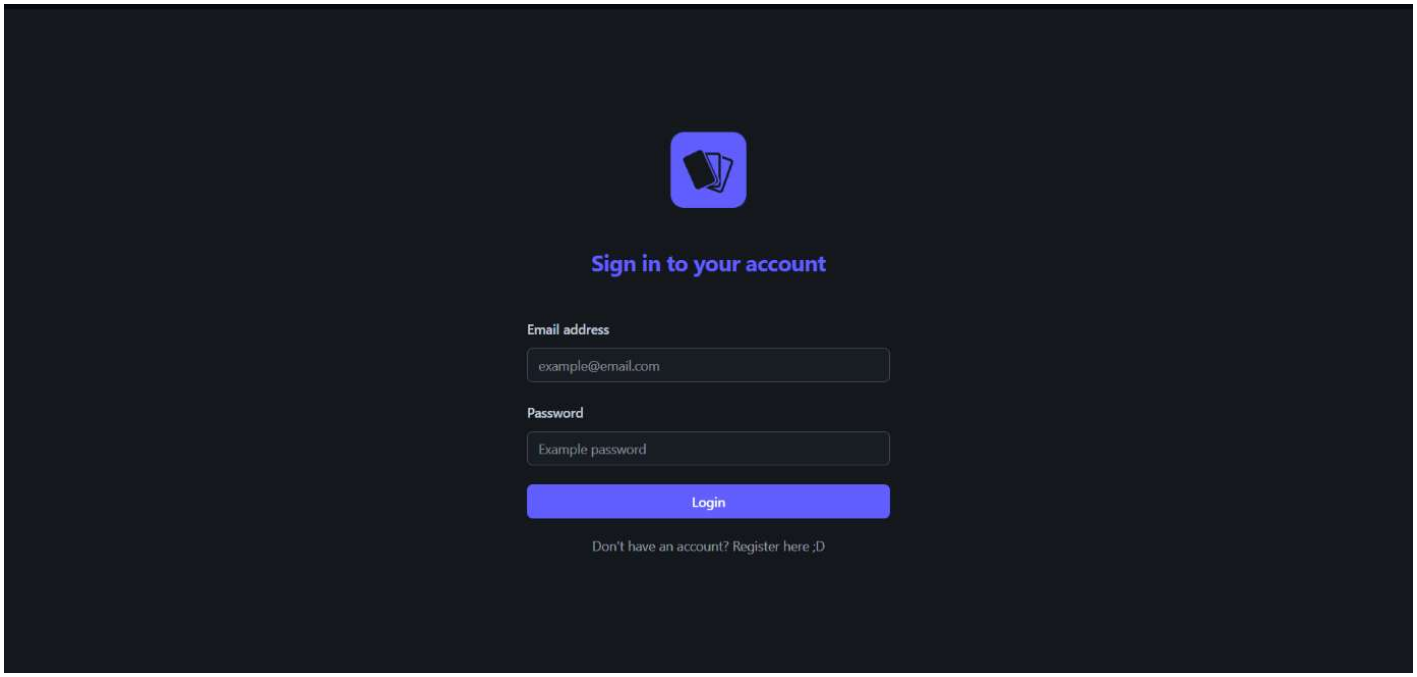
1. Regisztráció és Bejelentkezés

Az alkalmazás használatának első lépése egy privát fiók létrehozása. Új felhasználó a regisztrációs űrlapon e-mail cím, jelszó és egy megjelenített név megadásával hozhat létre fiókot. Sikeres regisztráció után a bejelentkezési képernyőn léphet be a rendszerbe.

Regisztráció:

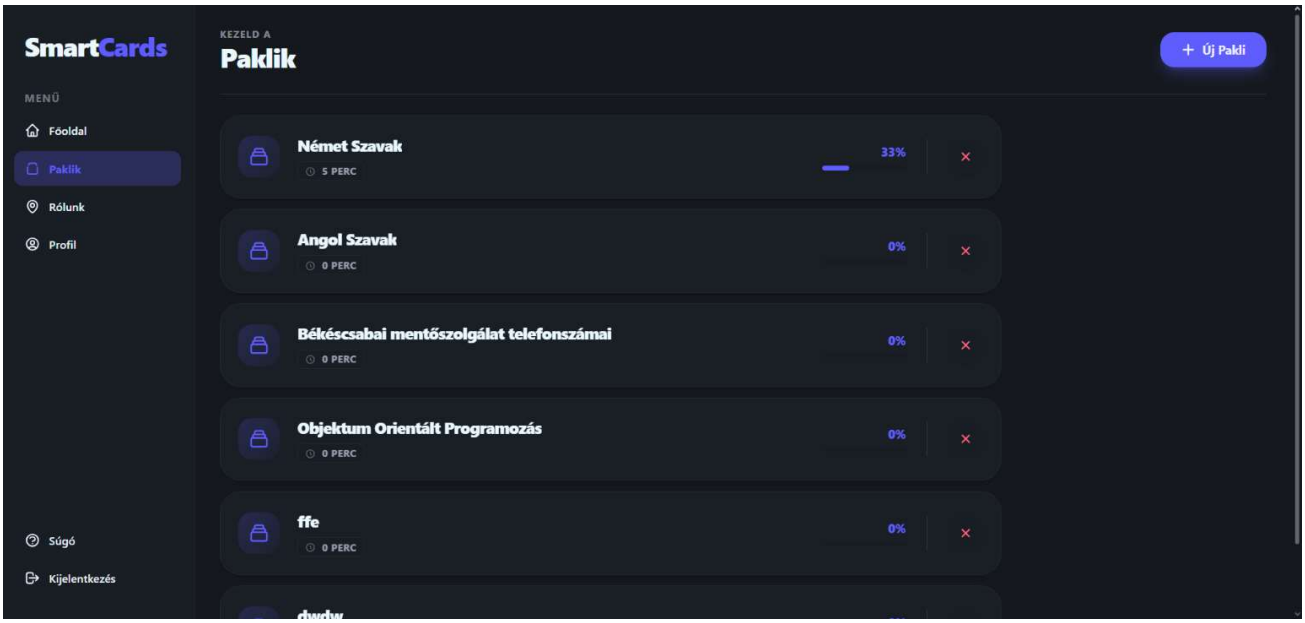


Bejelentkezés:

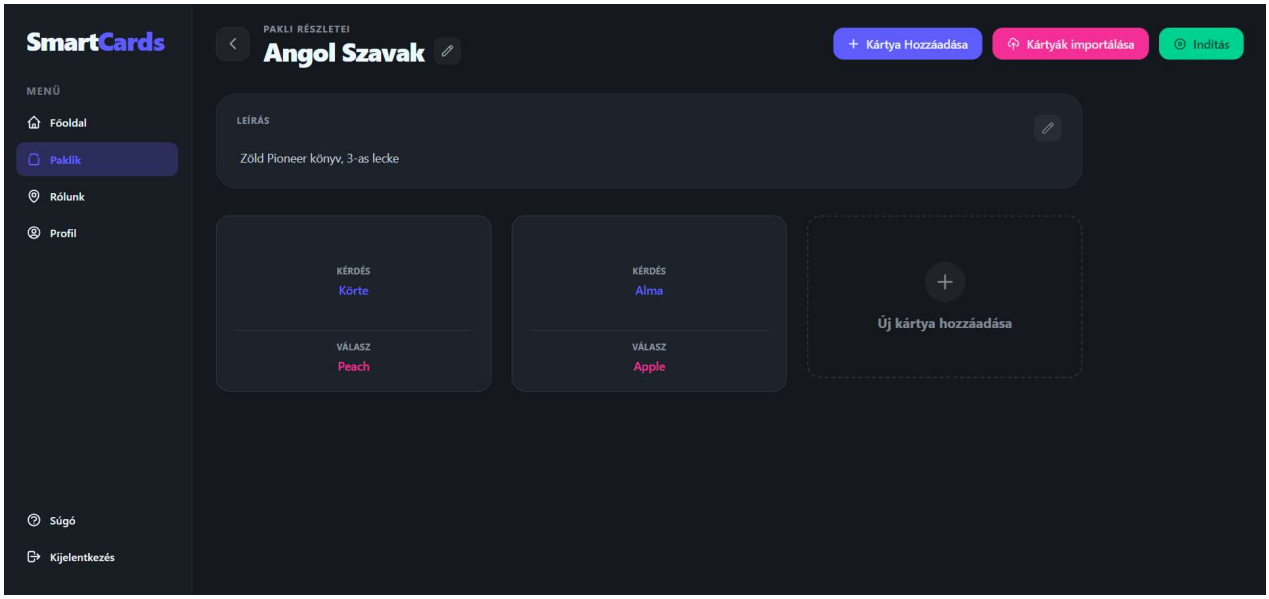


2. Új pakli és kártyák létrehozása

A felhasználó a "Paklik" menüpontban a "Új pakli" gombra kattintva indíthatja el a folyamatot. Például létrehozhat egy "Angol szavak" nevű csomagot.



A pakli megnyitása után az "Add Card" (Kártya hozzáadása) funkcióval viheti fel a kérdés-válasz párokat (pl. Kérdés: Apple, Válasz: Alma). Nagyobb mennyiségű anyag esetén lehetőség van tömeges szöveges betöltésre (Bulk Import) is.

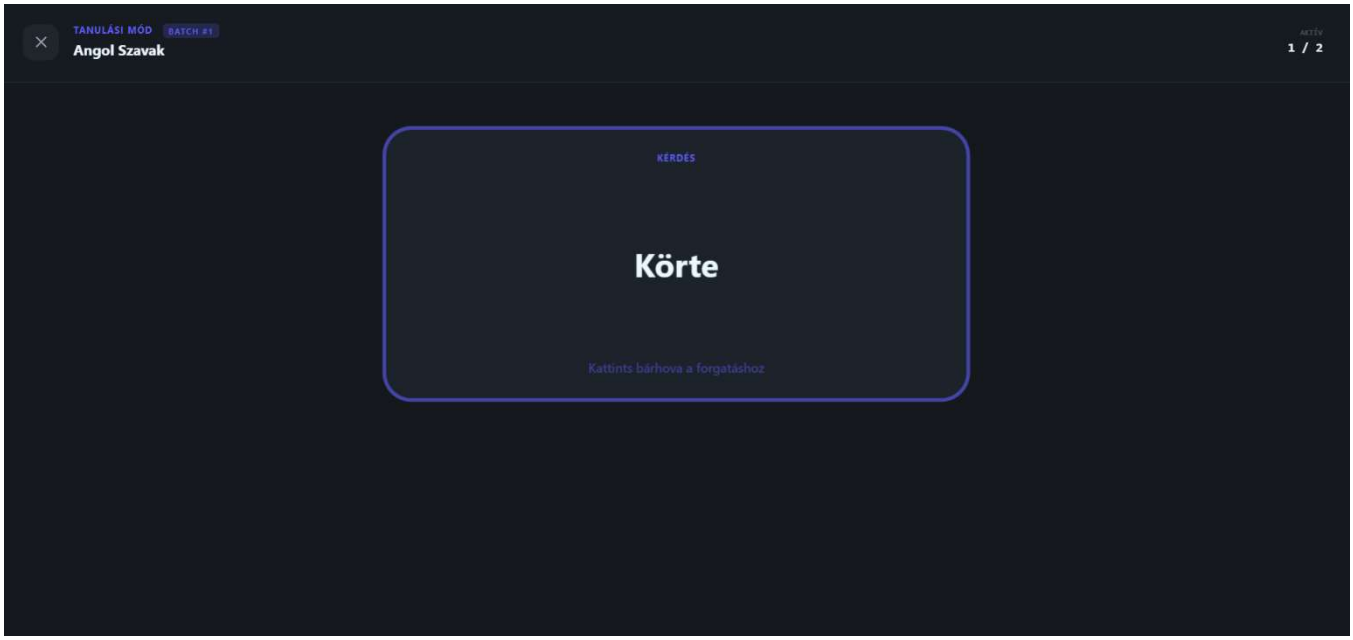


3. A tanulási munkamenet elindítása

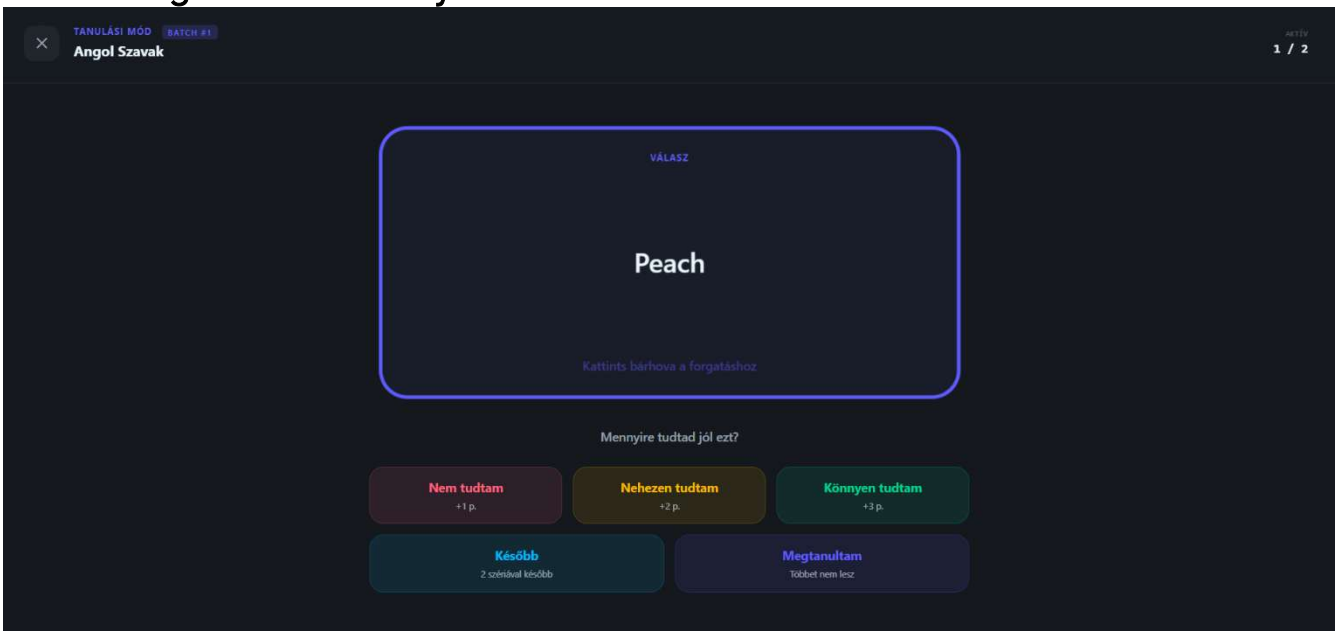
A tanulás a pakli oldalán található "Indítás" gombbal kezdődik.

- A képernyőn megjelenik az első kártya kérdés oldala.
- A felhasználó magában megválaszolja a kérdést, majd a kártyára (vagy a gombra) kattintva a kártya animálva megfordúl, felfedve a helyes választ.
- Ezt követően két gomb jelenik meg az önértékeléshez: "Nem tudtam" (Need review) vagy "Tudtam" (Mastered). A rendszer ez alapján számolja majd a százalékos statisztikákat.

Első kártya eleje:



Megfordított kártya:



4. Statisztikák és fejlődés nyomon követése

Amikor a felhasználó a pakli végére ér, a rendszer beküldi az eredményeket a szervernek. Visszatérve a Főoldalra, a felhasználó azonnali visszajelzést kap:

- Növekszik az eheti tanulással töltött idejét mutató oszlopdiagram.
- Ha aznap még nem tanult, megnő a napi "Streak" számlálója.
- A paklik listájában frissül a tudásszint érték az alapján, hogy mennyi kártyát jelölt meg elsajátítottként ("Megtanultam").

6. REST API Dokumentáció

I. Hitelesítés (AccountController)

- **POST /api/account/register:** Új felhasználó regisztrációja. Paraméterek (Body): Email, Password, DisplayName. Hozzáadja a kezdő UserStats-ot is az adatbázisban.
- **POST /api/account/login**: Bejelentkezés. Visszaadja a JWT tokent és beállítja a refreshToken HTTP-Only cookie-t.
- **POST /api/account/refresh-token**: Új JWT token igénylése a lejáratlan frissítő token alapján.
- **POST /api/account/change-password**: Jelszó módosítása (Auth szükséges).

II. Paklik Kezelése (DecksController)

Autentikáció szükséges

- **GET /api/decks**: A bejelentkezett felhasználó paklijainak lekérése.
- **GET /api/decks/last-played?limit=5**: A legutóbb játszott paklik listázása.
- **GET /api/decks/{id}/for-game**: Egy konkrét pakli és a hozzá tartozó összes kártya lekérése a játékmenethez.
- **POST /api/decks**: Új pakli létrehozása. Paraméterek (Body): Title, Description, Goal.
- **PUT /api/decks/{id}**: Pakli metaadatainak frissítése.
- **DELETE /api/decks/{id}**: Pakli törlése. Frissíti a globális kártya és pakli számlálókat (TotalDecks, TotalCards).
- **POST /api/decks/sync**: Offline mód vagy kliens oldali szinkronizáció támogatása (hozzáadott, módosított, törölt paklik listájával).

III. Kártyák Kezelése (CardsController)

- **POST /api/decks/{deckId}/cards**: Új kártya létrehozása egy pakliban. Paraméterek (Body): Question, Answer.
- **PUT /api/decks/{deckId}/cards/{id}**: Kártya módosítása.
- **DELETE /api/decks/{deckId}/cards/{id}**: Kártya törlése.
- **POST /api/decks/{deckId}/cards/import**: Bulk import támogatás (több kártya felvitele egyszerre szövegből).
- **POST /api/decks/{deckId}/cards/sync**: Kártyák szinkronizálása kliens és szerver között.

IV. Statisztikák (StatsController)

- **GET /api/stats/home**: A felhasználó kezdőlapra szánt statisztikáinak lekérése.
- **POST /api/stats/session**: Tanulási munkamenet erednényeinek elmentése.

V. Tanulási Statisztikák Matematikai és Logikai Összegzése

A rendszer komplex statisztikakövetést alkalmaz, amelyet a StatsService kezel:

1. **Tanulási Sorozat (Learning Streak):**
    - Minden munkamenet mentésekor vizsgálja a LastFlipAt (legutolsó tanulás ideje) értékét.
    - Ha a legutóbbi tanulás tegnap történt (daysPassed == 1), a LearningStreak értéke nő +1-gyel.
    - Ha több mint 1 nap telt el, a széria megszakad, és az érték 0-ra vagy 1-re áll vissza.
2. **Heti Aktivitás (WeeklyActivityJson):**

- Egy 7 elemű tömbben (hétfőtől vasárnapig) tárolja a naponta tanulással eltöltött perceket.
- **Gördülő Nullázás:** A rendszer kiszámítja az eltelt napokat a legutóbbi munkamenet óta. Ha 7 vagy több nap telt el, az egész tömböt lenullázza. Ha kevesebb, akkor csak a kimaradt napok indexeit állítja 0-ra.
- A friss munkamenet idejét (percben) hozzáadja az aktuális nap indexéhez (Math.Max(0, updateDto.TimeSpentMinutes)).

3. **Pakli Tudásszintje:**
    - Matematikai formula: (MasteredCards / TotalCardsInDeck) * 100
    - Lekerekítve egész számra. Példa: 10 kártyából 4 "Mastered" állapotú -> 40%.
4. **Főoldali statisztikák folyamatos frissítése:**
    - A Vezérlőpult (HomePage) folyamatos szinkronban van a tanulási eredményekkel, ami kritikus a gamifikáció (motiváció) fenntartásához.
    - Amikor egy tanulási munkamenet befejeződik, a frontend egy aszinkron HTTP POST kéréssel felküldi a friss eredményeket (Batch Request).
    - Sikeres válasz (HTTP 200 OK) esetén a kliens alkalmazás azonnal, oldalfrissítés (Reload) nélkül frissíti a helyi memóriában tárolt állapotot (State). Az alkalmazás reaktív adatfolyamot (pl. RxJS BehaviorSubject vagy React Context) használ, így a változás detektálása után a Dashboard komponensei automatikusan újrarajzolódnak, és a felhasználó azonnal látja a megnövekedett "Streak" ikonját vagy a frissített tudásszázalékokat.
5. **Heti jelentés diagram logikája**:
    - A heti jelentés mechanikája a kliens és a szerver szoros együttműködésére épül.

- A WeeklyActivityJson mezőt az API generálja le és tartja karban (a rolling-reset logikával).
- Ezt a 7 számjegyből álló tömböt a szerver nyers adatként küldi le a frontend számára.
- A kliens oldalon ezt az adatot egy vizuális diagram komponens fogadja be (amely lehet natív CSS/HTML oszlopdiagram, vagy egy külső könyvtár). A frontend kiszámítja az oszlopok maximális magasságát (a héten legtöbbet tanult nappercéhez viszonyítva), majd arányosan méretezi és animálva rajzolja ki a hét napjait jelképező sávokat.

7. Frontend Komponensek

I. Fő Oldalak:

- **HomePage:** Áttekintő dashboard a napi aktivitás charttal, streak counterrel és a legutóbb használt paklikkal.
- **AboutPage:** Ez a felület kettős célt szolgál. Egyrészt a vizsgakövetelményeknek megfelelően itt kapott helyet a projekt hivatalos leírása, a felhasznált keretrendszerek ismertetése és a készítők (fejlesztőcsapat) bemutatkozása.
- **DeckPage:** Paklik böngészése, keresése, új pakli felvitele.
- **ProfilePage:** Itt lehet felhasználói adatokat, az idézetek tpusát, témát és nyelvet változtatni.
- **DeckDetails / Editor:** Pakli tartalmának szerkesztése, kártyák hozzáadása.
- **FlashcardQuiz (Game):** Maga az interaktív tanulófelület. Kezeli a kártyák forgatását, a pontozást (IsMastered kalkuláció) kliens oldalon, majd a végén egyben (batch) küldi be a szervernek az eredményeket.

- **Állapotkezelés:** API hívások kezelése szerviz osztályokon keresztül (pl. game.service.ts), Interceptorok a JWT token csatolására és hibaüzenetek (pl. 401 Unauthorized) lekezelésére.
- **Megjelenés:** Tailwind CSS (vagy ahhoz hasonló utility-first keretrendszer) használata, sötét mód (Dark Mode) támogatás, reszponzív Grid és Flexbox elrendezések.

II. Kártya mechanika és vizuális interakció

- A tanulókártyák (Flashcardok) kezelése az alkalmazás legkritikusabb funkciója, amely a szerveroldali adatelérés és a kliensoldali megjelenítés precíz együttműködésére épül.
- **Kártyák sorrendje és Adatelérés (Backend logika):** Amikor a kliens elindít egy tanulási munkamenetet (Game), a rendszer a szerveroldali GetDeckForGameAsync repository metódust hívja meg. Az Entity Franework Core lekérdezés a paklihoz tartozó kártyákat szigorúan a létrehozásuk ideje alapján, csökkenő sorrendben rendezi sorba (OrderByDescending(c => c.CreatedAt)). Ez azt jelenti, hogy a tanulás során a felhasználó mindig a legfrissebben felvett (legujabb) kártyákkal találkozik először. A lekérdezés a kártyákhoz tartozó egyéni statisztikákat (CardStats) is azonnal hozzáfűzi, ráadásul a teljesítmény maximalizálása érdekében a memóriafogyasztó Descartes-szorzat elkerülésével, osztott lekérdezésként (AsSplitQuery) hajtja végre az SQL parancsot.
- **Vizuális elrendezés (Frontend Layout):** A szervertől megkapott kártyatömböt a kliens a képernyő közepére igazítja modern CSS technológiák (Flexbox és CSS Grid) segítségével. A kártyák úgynevezett "stak" (pakli) logikával jelennek meg: a memóriában lévő tömböt egy belső állapotgép (State Machine) vezérli, amely az aktuális index alapján mindig csak a legfelső kártyát teszi láthatóvá. A reszponzív kialakítás garantálja a tökéletes,

figyelemelterelésmentes megjelenítést mind vertikális mobil, mind horizontális asztali környezetben.

- **Animációk és 3D tér (Mechanika):** A kártya megfordítása (Flip) nem pusztán kliens oldali adatcsere, hanem egy valós idejű CSS 3D tranzformáció (rotateY(180deg)). A kártya HTML felépítése két rétegből (egy előlap és egy hátlap) áll, amelyek háttal vannak egymásnak fordítva (backface-visibility: hidden;). Kattintáskor a szülő konténer átfordul, így egy valósághű fizikai kártyaforgatás illúzióját nyújtja, miközben alul megjelennek az önértékelő (Tudtam / Nem tudtam) gombok.

III. A folyamatos adattárolás és betöltés:

- **Aszinkron** állapotkezelés és Vizuális Visszajelzés (Loading UI): Mivel a kliens-szerver kommunikáció az internetkapcsolattól függően időt vehet igénybe, a fromtend robusztus aszinkron állapotkezeléssel lett felvértezve.
  Minden hálózati kérés (HTTP Request) indításakor az adott komponens állapota "Betöltés" (Loading) státuszba vált.
  Ekkor jelenik meg a felületen a "Loading" animáció (Spinner).
  Ez a mechanika megakadályozza, hogy a felhasználó többször is rákattintson egy gombra, megelőzve az adatok kétszeres beküldését vagy a kifagyás érzetét.

- Adat-Cachelés (Caching) és Teljesítményoptimalizálás:

  Szerver oldalon: Annak érdekében, hogy a Főoldal (Dashboard) villámgyorsan betöltsön, a bonyolult SQL COUNT() aggregációs lekérdezéseket elkerültük.
  Ehelyett a paklik és kártyák összesített számát a UserStats táblában (TotalDecks, TotalCards) tartjuk gyorsítótárazvva (cache-elve). Így a teljes Főoldal felépítése egyetlen, gyors adatbázis lekérdezéssel kiszolgálható.

- **Kliens oldalon:** A frontend szolgáltatásai a már letöltött adatokat (például a paklik listáját) lokális memóriában (Cache) tárolják. Visszanavigáláskor az alkalmazás először a cache-ből jeleníti

meg a tartalmat azonnali betöltést produkálva, miközben a háttérben (silently) egy lekérdezéssel ellenőrzi a szervert, hogy történt-e azóta adatváltozás. Ez a mechanika minimalizálja a hálózati forgalmat, és professzionális, akadásmentes élményt biztosít a felhasználónak.

IV. Globális Állapotok és UI testreszabhatóság

- A modern webalkalmazásokkal szemben alapvető elvárás a magas fokú személyre szabhatóság. A **SmartCards** ezt a témák és a nyelv azonnali, dinamikus cseréjével biztosítja.
- **Témaváltás:** A vizuális megjelenés testreszabása a **Tailwind CSS** és a **DaisyUI** keretrendszer szoros integrációjára épül. Amikor a felhasználó megváltoztatja a témát (pl. Világosról Sötét módra), a frontend egyetlen kattintásra módosítja a gyökér HTML elem data-theme attribútumát. Mivel a rendszer CSS változókat (CSS variables) használ a fix színkódok (hex/rgb) helyett, az attribútum cseréje oldalfrissítés nélkül, azonnal átszínezi a teljes alkalmazást (hátterek, gombok, szövegek).
- **Nyelvváltás (Internationalization / i18n):** Az alkalmazás többnyelvűségét a kliensoldalon egy központi nyelvi szótár (állapottér) kezeli. A szöveges tartalmak (gombok feliratai, hibaüzenetek, menüpontok) nem a HTML kódban vannak "beégetve" (hardcoded), hanem változókon keresztül töltődnek be a nyelvi fájlokból. Nyelvváltás esetén a frontend globális állapota frissül, így a reaktív adatkötés (Data Binding) révén a képernyőn lévő összes felirat a másodperc törtrésze alatt átvált a kiválasztott nyelvre, megszakítás nélkül folytatva a felhasználói élményt.

8. Részletes Tesztelési Stratégia

I. Manuális Tesztek és Várt Eredmények

Teszteset	Művelet / Bemeneti Adat	Várt Eredmény
Auth 01	Regisztráció: Email:example11111@gmail.com Jelszó:Ex@mple11111, Név: ExampleUser	200 OK, Létrejön a fiók, és visszatér a JWT Token. Bedob a Főoldalra
Auth 02	Bejelentkezés hibás jelszóval: Email:example11111@gmail.com, Jelszó: WrongPw123	401 Unauthorized ("Invalid password" hibaüzenet).
Deck 01	Új pakli (Body: {"Title": "Angol szavak", "Goal": "daily"})	200 OK, Létrejön az adatbázisban, visszakapjuk a DeckDto-t guid Id-val.
Card 01	Új kártya létrehozása bemásolással: Apple;Alma Peach;Körte	201 Created, Kártya rögzítve a megfelelő paklihoz.
Sync 01	Tanulási munkamenet mentése (FlippedCardsToday: 10, TimeSpentMinutes: 5)	A StatsService frissíti a WeeklyActivity tömböt, megnő a Streak (ha tegnap is volt aktivitás).

Teszteset	Művelet / Bemeneti Adat	Várt Eredmény
Delete 01	Pakli törlése	A pakli törlődik az adatbázisból. A TotalDecks számláló csökken 1-gyel a UserStats táblában. A Cascade miatt a kártyák is törlődnek.

II. Automatizált Tesztek

- **Unit Tesztek:** C# szinten a service-k matematikai logikájának tesztelése xUnit keretrendszer segítségével.

```
  API.Tests net10.0 succeeded (0,1s) → bin\Debug\net10.0\API.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.5)
[xUnit.net 00:00:00.05]   Discovering: API.Tests
[xUnit.net 00:00:00.08]   Discovered:  API.Tests
[xUnit.net 00:00:00.10]   Starting:    API.Tests
     Warning:
     The component "Fluent Assertions" is governed by the rules defined in the Xceed License Agreement and
     the Xceed Fluent Assertions Community License. You may use Fluent Assertions free of charge for
     non-commercial use only. An active subscription is required to use Fluent Assertions for commercial use.
     Please contact Xceed Sales mailto:sales@xceed.com to acquire a subscription at a very low cost.
     A paid commercial license supports the development and continued increasing support of
     Fluent Assertions users under both commercial and community licenses. Help us
     keep Fluent Assertions at the forefront of unit testing.
     For more information, visit https://xceed.com/products/unit-testing/fluent-assertions/
[xUnit.net 00:00:00.22]   Finished:    API.Tests
  API.Tests test net10.0 succeeded (0,7s)

Test summary: total: 12; failed: 0; succeeded: 12; skipped: 0; duration: 0,7s
Build succeeded in 1,6s
```

- **Vitest tesztek:** Angular szinten a kliensoldali logika megbízhatóságát Vitest keretrendszerrel végzett egységtesztek biztosítják.

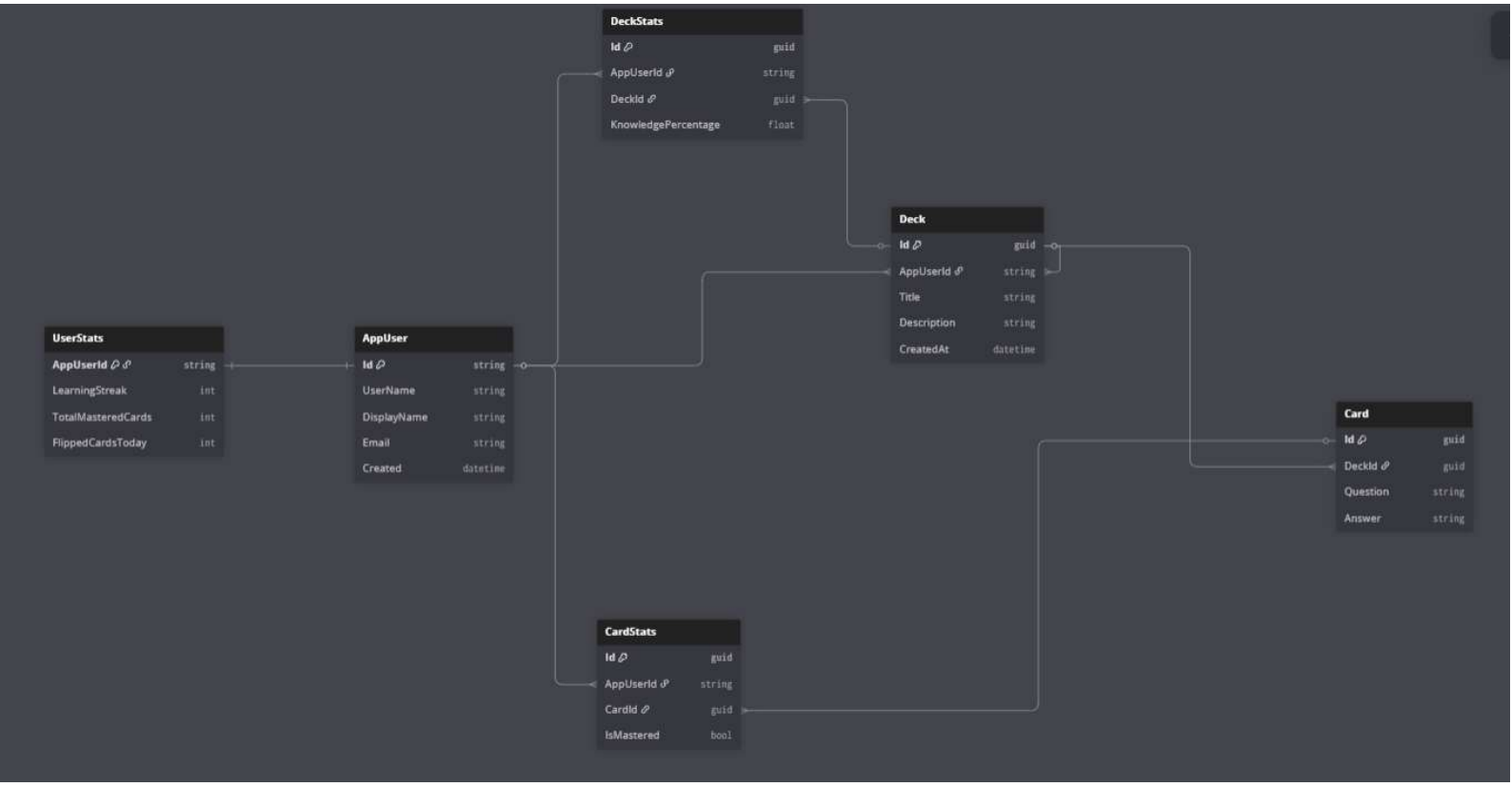
```
 ✓ client  src/core/services/busy-service.spec.ts (4 tests) 18ms
 ✓ client  src/core/services/toast-service.spec.ts (3 tests) 54ms
 ✓ client  src/core/services/home-service.spec.ts (4 tests) 39ms
 ✓ client  src/core/i18n/language-service.spec.ts (4 tests) 18ms
 ✓ client  src/core/services/game-service.spec.ts (4 tests) 25ms
 ✓ client  src/core/services/deck-service.spec.ts (2 tests) 34ms
 ✓ client  src/features/game/game.spec.ts (5 tests) 65ms
stdout | src/features/decks/deck-detail/deck-detail.spec.ts > DeckDetailComponent > should temporarily add a card when onCreateCard is called
Syncing cards... {
  addedCards: [
    {
      question: 'New Q',
      answer: 'New A',
      tempId: 'temp-0.2702309101306271 7'
    }
  ],
  updatedCards: [],
  deletedCardIds: []
}

 ✓ client  src/features/decks/deck-detail/deck-detail.spec.ts (3 tests) 87ms

 Test Files  8 passed (8)
      Tests  29 passed (29)
   Start at  23:19:38
   Duration  1.23s (transform 480ms, setup 2.22s, import 776ms, tests 340ms, environment 4.27s)

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

9. Adatbázis Dokumentáció

A **SmartCards** alkalmazás egy relációs adatbázist használ, amelynek struktúráját és kapcsolatait az Entity Framework Core ORM (Object-Relational Mapping) szoftveres rétege kezeli, Code-First megközelítéssel. Az adatok integritásának fenntartása érdekében szigorú idegen kulcs és törlési szabályok kerültek implementálásra.

I. Adatbázis Modell és ER Diagram

Az alábbi egyed-kapcsolat (ER = Entity-Relationship) diagramm szemlélteti a rendszerben található táblákat, a bennük tárolt főbb attribútumokat és a táblák közötti kardinalitást.



10. Adatbázis Séma és Kapcsolatok

A rendszer 6 dedikált fő entitással (táblával) dolgozik, emellett az AppUser az ASP.NET Core Identity rendszerébe (az AspNetUsers táblába) épül be. A táblakapcsolatok konfigurálása (Fluent API) a DataContext fájlban történt.

I. AppUser (Felhasználói fiók)

Az identitást és bejelentkezést kezelő tábla.

- **Típus:** IdentityUser kiterjesztés.
- **Mezők:** Id (PK), DisplayName, Email, RefreshToken, RefreshTokenExpiry, Created (Alapértelmezés: DateTime.UtcNow).
- **Kapcsolatok:**
- **1 : N** kapcsolat a Deck táblával. (Törlés: Cascade – Ha a felhasználó törlődik, az összes paklija eltűnik).
- **1 : 1** kapcsolat a UserStats táblával. (Törlés: Cascade).
- **1 : N** kapcsolat a DeckStats táblával. (Törlés: Restrict – Biztonsági okokból nem engedélyezi a direkt törlést, amíg statisztikák vannak).
- **1 : N** kapcsolat a CardStats táblával. (Törlés: Restrict).

II. Deck (Kártyacsomag / Pakli)

Egy témakörbe tartozó tanulókártyák gyűjtőentitása.

- **Mezők:**
  - Id (Guid, PK)
  - AppUserId (String, FK - Required)
  - Title (String, Required, Max length: 255)
  - Description (String, Opcionális)
  - CreatedAt (DateTime)
- **Kapcsolatok:**

- **1 : N** kapcsolat a Card táblával. (Törlés: Cascade – A pakli törlésével az összes alatta lévő kártya is törlődik az adatbázisból).
- **1 : N** kapcsolat a DeckStats táblával. (Törlés: Cascade).

III. Card (Tanulókártya / Flashcard)

Az "előlap" és "hátlap" adatokat tartalmazó entitás.

- **Mezők:**
  - Id (Guid, PK)
  - DeckId (Guid, FK - Required)
  - Question (String, Required - A kártya kérdése/előlapja)
  - Answer (String, Required - A kártya válasza/hátlapja)
  - CreatedAt (DateTime)
- **Kapcsolatok:**
  - **1 : N** kapcsolat a CardStats táblával. (Törlés: Cascade – A kártya törlésével annak egyéni tanulási statisztikái is elvesznek).

IV. UserStats (Globális Felhasználói Statisztika)

Egy az egyhez kapcsolatban áll a felhasználóval, az összesített eredményeket tárolja. A PK és FK megegyezik (AppUserId).

- **Mezők:**
- AppUserId (String, PK, FK - Required)
- FlippedCardsTotal (Long) – Összes eddigi forgatás.
- FlippedCardsToday (Int) – Napi forgatások.
- LearningStreak (Int) – Megszakítás nélküli tanulási napok száma.
- TotalDecks (Int) & TotalCards (Long) – Paklik és kártyák számának gyorsítótárazott (cached) értéke a lekérdezések gyorsítására.
- TotalMasteredCards (Long) – Elsajátított kártyák száma.
- LastFlipAt (DateTime) – Utolsó aktivitás időbélyege.
- WeeklyActivityJson (String) – JSON tömbben tárolt percenkénti aktivitás heti lebontásban (pl. [0, 15, 0, 45, 0, 0, 0]).

V. DeckStats (Pakli specifikus statisztikák)

Egy adott felhasználó adott paklijához tartozó elért eredményeit rögzíti.

- **Mezők:**
- Id (Long, PK)
- AppUserId (String, FK - Required)
- DeckId (Guid, FK - Required)
- KnowledgePercentage (Int) – Hány százalékát tanulta meg a paklinak (0-100%).

- TimeSpentMinutes (Int) – Összesen a paklival eltöltött tanulási idő percekben.
- LastPlayedAt (DateTime) – Mikor indított a pakliból kvízt utoljára.
- Goal (String) – Célkitűzés (pl. "Napi").

VI. CardStats (Kártya specifikus tanulási állapot)

Lehetővé teszi, hogy a rendszer tudja: egy bizonyos kártyát a felhasználó épp milyen fázisban tanul.

- **Mezők:**
- Id (Long, PK)
- AppUserId (String, FK - Required)
- CardId (Guid, FK - Required)
- BatchIndex, RotationPoints, RotationIndex (Int) – Belső logika a spaced repetition vagy egyéni forgatási logoritmushoz.
- IsMastered (Bool) – Igaz, ha a felhasználó a kártyát "Tudtam" állapotba helyezte.

11. Tesztelés

A **SmartCards** alkalmazás megbízhatóságának és stabilitásának garantálása érdekében a fejlesztés során mind a backend, mind a frontend rétegben szigorú tesztelési irányelveket követtünk.

Tesztelési terv áttekintése

A tesztelési stratégia több szintre bontható:

- **Egységtesztelés (Unit Testing):** A szerveroldali üzleti logika (pl. a StatsService statisztikai algoritmusai) tesztelése, ahol a metódusok izoláltan kerülnek vizsgáltatásra az adatelérési réteg (Unit of Work) "kigúnyolásával"(Mockolásával/helyettesítésével) (Mocking, pl. Moq keretrendszerrel).
- Frontend egységtesztek (Vitest): A kliensoldali logika megbízhatóságát Vitest keretrendszerrel végzett egységtesztek biztosítják. A tesztelési folyamat kiterjed a projekt minőségbiztosítása során a kritikus üzleti logikára és az adatszolgáltató rétegre. Ez garantálja, hogy a felhasználói felület elemei a várt módon reagálnak a különböző bemeneti adatokra.
- **Végpont tesztelés (API/Integration Testing):** A RESTful API végpontok (Controllers) ellenőrzése Postman kollekciók és Swagger segítségével. Itt azt vizsgáltuk, hogy megfelelő kérésre a helyes HTTP státuszkódok és JSON DTO-k érkeznek-e válaszként.
- **Felhasználói felület és End-to-End tesztelés (Manual E2E Testing):** A teljes rendszer integrált működésének manuális tesztelése a böngészőn keresztül. Ebbe beletartozik a UI elemek reszponzivitása és a teljes tanulási folyamat (regisztrációtól a kvíz befejezéséig) lefedése.

VII. Egységtesztek áttekintése és példák

- Az egységtesztek (Unit Tesztek) leginkább a kritikus üzleti logikát tartalmazó funkciókra fókuszáltak, az xUnit keretrendszer használatával. Kiemelten fontos volt a **tanulási sorozat (Learning Streak)** matematikai logikájának ellenőrzése.**Példa egy kulcsfunkció (StatsService) tesztelésére:** A SaveStudySessionAsync metódusnak megfelelően kell növelnie a "Streak" értékét, ha a felhasználó tegnap is tanult.

```
[Fact]
    public async Task
GetUserStatsAsync_WhenExactlyOneDayPassed_IncreasesStreak()
    {
        // Arrange
        var userId = "test-user-id";
        var today = DateTime.Now.Date;
        var yesterday = today.AddDays(-1);

        var userStats = new UserStats
        {
            AppUserId = userId,
            LastFlipAt = yesterday,
            LearningStreak = 5,
            FlippedCardsToday = 20
        };

        _mockUnitOfWork.Setup(u =>
u.StatsRepository.GetUserStatsAsync(userId))
            .ReturnsAsync(userStats);

        _mockMapper.Setup(m => m.Map<UserStatsDto>(It.IsAny<UserStats>()))
            .Returns((UserStats source) => new UserStatsDto
            {
                LearningStreak = source.LearningStreak
            });

        // Act
        var result = await _statsService.GetUserStatsAsync(userId);

        // Assert
        result.LearningStreak.Should().Be(6); // Streak should increment by
1
        _mockUnitOfWork.Verify(u => u.Complete(), Times.Once);
    }
```

VIII. Manuális és Automatizált Tesztesetek, Várt Eredmények

Az alábbi tábllázat tartalmazza az API és a Kliens oldal integrált tesztelésének legfontosabb eseteit, amelyeket manuálisan (illetve Postman Automation segítségével) futtattunk le.

TC_AUTH_01	Hitelesítés	Regisztráció érvényes, de már regisztrált e-mail címmel.	HTTP 400 Bad Request, "Email is already taken" vagy hasonló validációs hibaüzenet.	Pass
TC_AUTH_02	Hitelesítés	Regisztráció túl rövid / gyenge jelszóval (pl. "123").	HTTP 400 Bad Request. A backend regex validátora elutasítja a jelszót (Minimum hossz/szám/nagybetű hiánya miatt).	Pass
TC_AUTH_03	Hitelesítés	Bejelentkezés érvényes adatokkal.	HTTP 200 OK. A válasz tartalmazza a JWT Token-t, a HTTP-Only Cookie-ban pedig a Refresh Tokent.	Pass
TC_AUTH_04	Hitelesítés	Refresh Token végpont hívása lejárt JWT token esetén.	HTTP 200 OK. A rendszer érvényesíti a sütit, és új, érvényes JWT tokent generál megszakítás nélkül.	Pass
TC_DECK_01	Paklik	Végpont hívása Authorization Token nélkül (GET /api/decks).	HTTP 401 Unauthorized. Az azonosítatlan hozzáférés elutasítva.	Pass
TC_DECK_02	Paklik	Új pakli létrehozása (Body: Title="Új pakli", Goal="Napi").	HTTP 200 OK. A pakli létrejön, beállítódik hozzá az aktuális User ID-ja és egy Guid azonosító.	Pass
TC_DECK_03	Paklik	Pakli létrehozása hiányzó Title paraméterrel.	HTTP 400 Bad Request. A DTO Data Annotations validációja hibát dob.	Pass
TC_DECK_04	Adatbázis	Pakli törlése (Cascade ellenőrzés).	HTTP 200 OK. A pakli és a hozzá tartozó összes	Pass

			kártya (Cards) és statisztika (DeckStats) törlődik. A UserStats TotalDecks számlálója csökken.	
TC_SEC_01	Biztonság	Felhasználó (A) megpróbálja lekérni Felhasználó (B) pakliját az ID módosításával a URL-ben.	HTTP 404 Not Found (vagy 403 Forbidden). A lekérdezés tartalmazza az AppUserId == userId szűrőt, így nem találja meg.	Pass
TC_CARD_01	Kártyák	Új kártya hozzáadása üres Question mezővel.	HTTP 400 Bad Request. A kérdés mező kötelező (Required).	Pass
TC_CARD_02	Kártyák	"Bulk Import" tesztelése helyes formátumú szöveggel (Kérdés;Válasz \n ...).	HTTP 200/201. A kártyák kötegelve létrejönnek a pakli alatt.	Pass
TC_CARD_03	Kártyák	"Bulk Import" tesztelése hibás formátumú (hiányzó elválasztójel) szöveggel.	HTTP 400 Bad Request. A parser lekezeli a hibát, és jelzi, melyik sor a hibás. Nincs részleges mentés.	Pass
TC_SYNC_01	Szinkron	Kártyák eredményének beküldése (POST /api/decks/sync) batch módosításként.	HTTP 200 OK. A szerver feldolgozza a DTO tömböt, és frissíti a CardStats táblákat a Mastered értékekkel.	Pass
TC_STATS_01	Statisztika	Munkamenet beküldése negatív tanulási idővel.	HTTP 400 Bad Request (Validation hiba). A tanulási idő nem lehet kisebb 0-nál.	Pass
TC_STATS_02	Statisztika	Új tanulási szekció mentése a tegnapi tanulás után.	HTTP 200 OK. A LearningStreak értéke +1-gyel nő az adatbázisban.	Pass
TC_STATS_03	Statisztika	Tanulási szekció mentése 7+ nap kihagyás után.	HTTP 200 OK. A LearningStreak visszaugrik 1-re, és a WeeklyActivityJson tömb elemei "Rolling Reset" alapján nullázódnak.	Pass
TC_STATS_04	Statisztika	Tudásszázalék (Knowledge %) újraszámolás ellenőrzése.	A DeckStats táblában a Mastered kártyák aránya alapján helyesen frissül a százalék (pl. 2/10 -> 20%).	Pass
TC_UI_01	Fronten	Kártya forgatás	A kártya teljes egészében	Pas

	d	animáció (CSS rotateY) mobilos felbontásban (375px).	látszódik, a szöveg nem lóg ki (overflow hidden), a gombok tapinthatók.	s
TC_UI_02	Frontend	"Loading" Spinner ellenőrzése mesterségesen lassított (Throttled) hálózaton.	Amíg a HTTP kérés fut, a gombok inaktívak (disabled), és a betöltés jelző animáció hibátlanul forog.	Pass
TC_UI_03	Frontend	Sötét/Világos téma váltás (Theme Toggle).	Kattintáskor a data-theme azonnal kicserélődik, a CSS változók újrarenderezik az oldalt. Az állapot a LocalStorage-ba mentődik.	Pass
TC_UI_04	Frontend	Nyelvváltás (i18n).	Gombnyomásra a statikus feliratok, gombok és hibaüzenetek (pl. "Save" -> "Mentés") azonnal, oldalfrissítés nélkül átváltanak.	Pass
TC_AUTH_01	Hitelesítés	Regisztráció érvényes, de már regisztrált e-mail címmel.	HTTP 400 Bad Request, "Email is already taken"	Pass

12. Jövőbeli fejlesztési lehetőségek

A **SmartCards** alkalmazás a jelenlegi formájában egy stabil és jól működő alapot biztosít a hatákony tanuláshoz, azonban az architektúra (N-Tier és komponensalapú UI) kialakításának köszönhetően a rendszer rendkívül könnyen skálázható és bővíthető.

I. Funkcionális fejlesztési lehetőségek

Ezek a fejlesztések új funkciókat és közvetlen értéket nyújtanak a végfelhasználók számára:

- **Mesterséges Inteligencia integráció:** A jövőben tervezzük OpenAI vagy egyéb nyelvi modellek bekötését, amely segítségével a felhasználók hosszú szövegekből (pl. cikkek, könyvrészletek) vagy

feltöltött képekből (OCR technológiával) automatikusan, egy gombnyomásra generálhatnának kérdés-válasz kártyákat.

- **Közösségi funkciók és Piactér:** Jelenleg az adatelérés szigorúan privát (owner-based). Egy publikus megosztási funkcióval a felhasználók publikálhatnák a jól sikerűlt paklijaikat (pl. egyetemi tantárgyak tételeit), amelyeket mások letölthetnek, értékelhetnek vagy kiegészíthetnek egy Explore (Felfedezés) fül keretein belül.
- **Fejlett "Spaced Repetition" algoritmus:** A jelenlegi statisztikai rendszer kiegészíthető lenne egy tudományosan megalapozott időközönkénti ismétlő algoritmussal (pl. SuperMemo-2 vagy Anki algoritmusa), ami nemcsak a napi Streak-et méri, hanem prediktív módon kiszámolja, pontosan mikor fogja a felhasználó elfelejteni az adott kártyát, és csak akkor dobja fel ismétlésre.
- **Új játékmódok:** A hagyományos kártyaforgatás mellett tesztes (feleletválasztós), gépelős (helyesírási) vagy párosítós játékmódok bevezetése.

II. Nem funkcionális fejlesztési lehetőségek

Ezek a fejlesztések a rendszer teljesítményét, elérhetőségét és technikai minőségét javítják:

- **Natív Mobilalkalmazás:** Mivel a backend egy szabványos REST API, így lehetőség nyílik rendes mobil aplikációvá konvertálni és feltenni bármilyen hivatalos letöltő alkalmazásba (pl. Play Store/Apple Store)
- **Idősávos megoldás:** Továbbfejleszetni a jelenlegi statisztikai rendszert és külön algoritmusokat (gameket) megírni a napi, heti, és havi tanulásmódszerekhez.
- Felhasznált források

13. Telepítési és Üzemeltetési Útmutató

Ez a fejezet leírja a SmartCards alkalmazás elindításához szükséges lépéseket fejlesztői környezetben.

Előfeltételek:

A futtatáshoz a következő szoftverek telepítése szükséges:

- **.NET 8.0 SDK** (vagy magasabb)
- **Node.js** (LTS verzió) és **npm** csomagkezelő
- **Angular CLI** (`npm install -g @angular/cli`)

I. Backend indítása:

1. Navigáljon a projekt `Backend` mappájába.
2. Állítsa be az adatbázis elérhetőségét az `appsettings.json` fájlban (alapértelmezetten SQLite kerül felhasználásra).

```
dotnet ef database update
```

3. Futtassa az adatbázis migrációkat:

```
dotnet run
```

4. A REST API alapértelmezetten a http://localhost:5000 vagy https://localhost:5001 címen lesz elérhető.

II. Frontend indítása:

1. Navigáljon a projekt `Frontend(client)` mappájába.
2. Telepítse a szükséges függőségeket:

```
npm install
```

3. Indítsa el a fejlesztői szervert:

```
npm install
```

4. Nyissa meg a böngészőt a `http://localhost:4200`címen.

14. Felhasznált források

A projekt megtervezése és implementálása során számos dokumentációra, kurzusra és videóra támaszkodtunk. A használt technológiák (C# .NET, Enttity Framework, REST elvek, Frontend technológiák) tekintetében.

I. Hivatkozások és Linkek

- **Az Udemy kurzus:** Ez alapján indultunk neki az Angular + Typescript + Tailwind css + DaisyUI frontendnek. Megadta azt az összeszedett kezdőlökést amivel elindulhattunk a Full-stack fejlesztés irányába.
  Link: https://www.udemy.com/course/build-an-app-with-aspnet-core-and-angular-from-scratch/?couponCode=MT260421G3
- **Microsoft ASP.NET Core Dokumentáció:** A REST API felépítéséhez, az N-rétegű architektúra kialakításához és a Dependency Injection megértéséhez használt elsődleges forrás.
  Link: https://learn.microsoft.com/en-us/aspnet/core/
- **Entity Framework Core Dokumentáció:** Az adatbázis ORM beállításához, a Code-First migrációk és a relációs kapcsolatok (Cascade, Restrict) konfigurálásához.
  Linek: https://learn.microsoft.com/en-us/ef/core/
- **JSON Web Tokens (JWT) Bevezető:** A hitelesítési rendszer, a Stateless API kommunikáció és a biztonságos hálózati réteg (Auth) megvalósításának elméleti háttere.
  Link: https://jwt.io/introduction
- **Terjedelmes és tartalmas videó:** Megerősítette bennünk a modern Angular komponens alapú elvét, hogy ne teljes oldalak cserélgessünk kódduplikációval.
  Link: https://www.youtube.com/watch?v=oUmVFHlwZsI&t=4564s
- **Tailwind CSS Dokumentáció:** A "Utility-first" tervezési elvek alkalmazása, a reszponzivitás és a sötét mód (Dark Mode)

beállításának technikai megvalósítása a kliens oldalon.
Link: https://tailwindcss.com/docs

- **DaisyUi Dokumentáció:** Innen szedtük a komponenseket, ötleteket és mintákat a Tailwind css hatékony felhasználása érdekében, innen jött a témák ötlete majd meglepően egyszerű megvalósítása is.
  Link: https://daisyui.com/components/theme-controller/
- **HeroIcons használata:** A matematikai vektor alapú ikonokat (svg típusú „képek") innen szúrtuk be.
  Link: https://heroicons.com/solid
- **Clean Architecture & N-Tier Design (Blog cikkek és architektúra minták):** A Repository minta és a Service réteg elkülönítésének elméleti megalapozásához.
  Link:
  https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures
- **A Duális partnerünk forrásai:** A NIX-Duális partnercég teljes körű dokumentáltsága, és tananyagai felkészítettek minket az év elejétől kezdve.
  Link: https://edu.nixs.com/mod/h5pactivity/view.php?id=13363

15. Szerzői jogi nyilatkozat és Licenc

Proprietary License - SmartCards Project

A szoftver és annak teljes forráskódja nemzetközi szerzői jogi törvények és az **1999. évi LXXVI. szerzői jogi törvény** védelme alatt áll.

Copyright © 2026 Laczkó István & Brückner Gábor. Minden jog fenntartva.

A szerzők (Laczkó István és Brückner Gábor) fenntartanak minden jogot a szoftverrel és annak minden elemével kapcsolatban. A szoftver, vagy annak bármely részének jogosulatlan felhasználása, többszörözése, terjesztése vagy módosítása a szerzők előzetes írásbeli engedélye nélkül szigorúan tilos.